HG

AUTHOR · ROSHAN TRIVEDI

LAB TEST REPORT

HumanGate

MCP Human Approval Gateway

Can a person stay in control of what an AI agent is allowed to do? A hands-on test with a real MCP client, on a home laptop.

Real client: MCP Inspector v2.9.0 | Protocol 2025-11-25 | 34 of 34 tests pass | Synthetic tools only

Document ID	HG-MCP-TR-001
Version	1.0
Date	4 October 2026 (tests run 3 and 4 October 2026)
Author	Roshan Trivedi · IAM, PAM and agentic AI security
Licence	Code is MIT licensed. This report is © Roshan Trivedi.

roshantrivedi.co.in

LinkedIn · Roshan Trivedi

© 2026 Roshan Trivedi. All rights reserved.

Document control

Document	HumanGate MCP Human Approval Gateway: lab test report
ID and version	HG-MCP-TR-001 · v1.0
Status	Final for sharing, subject to the limits in section 5
Author	Roshan Trivedi
Test dates	3 October 2026 (tests T1 to T9) and 4 October 2026 (tests T10 to T14)
Source	github.com/trivediroshan1-ui/mcp-human-approval-gateway

At a glance

14 test cases

14 passed

9 synthetic tools

34/34 automated tests

Contents

1	Description and scope	3
2	Use case and flow	4
3	Test environment and method	5
T1	Install, build and start the gateway	6
T2	A real MCP client connects and lists the tools	7
T3	Low-risk request runs straight away	8
T4	Risky request is held for a human	9
T5	A security lead approves in the review screen	10
T6	The approved request runs exactly once	11
T7	Reusing the approval is refused	12
T8	Asking for extra permission is denied by policy	13
T9	Every step is in a tamper-evident log	14
T10	New test: fresh request held, then approved	15
T11	Same approval, changed target: refused	16
T12	The exact reviewed request still runs once	17
T13	The refusal is on the record	18
T14	The project's own automated tests	19
4	Results at a glance	20
5	Limits and what was not tested	20
6	What is next	21
7	About the author and contact	21
8	Copyright and notice	21

© 2026 Roshan Trivedi. All rights reserved.

1.1 Description

AI agents now call tools: search the docs, read a repository, change a role, deploy to production. Some of those calls matter. HumanGate is a gateway that sits between an agent and its tools and answers three questions before anything runs: **may this happen, who said yes, and can we prove it afterwards?**

1 Policy

Fixed rules score every call and decide: run, refuse, or hold for a person.

2 Human approval

A reviewer with the right role decides. An agent cannot approve its own request.

3 Bound, single use

The approval is tied to a hash of the exact request, expires quickly and works once.

4 Audit trail

Every decision goes into a hash-chained log that anyone can verify.

1.2 Scope

In scope

- Install and run the gateway on a home laptop
- A real MCP client (official Inspector v2.9.0) calling it over HTTP
- Low-risk call, risky call, human approval, single use
- Replay and changed-request attempts
- Over-scope request denied by policy
- Audit log shown in the lab page
- The project's automated MCP tests

Out of scope

- Any real production system: all nine tools are synthetic
- Real user login: this is an unauthenticated demo mode
- Clients other than the Inspector (Claude Code is next)
- An agent approving itself, tried by hand
- An approval expiring, tried by hand
- Signed audit log
- Performance and load

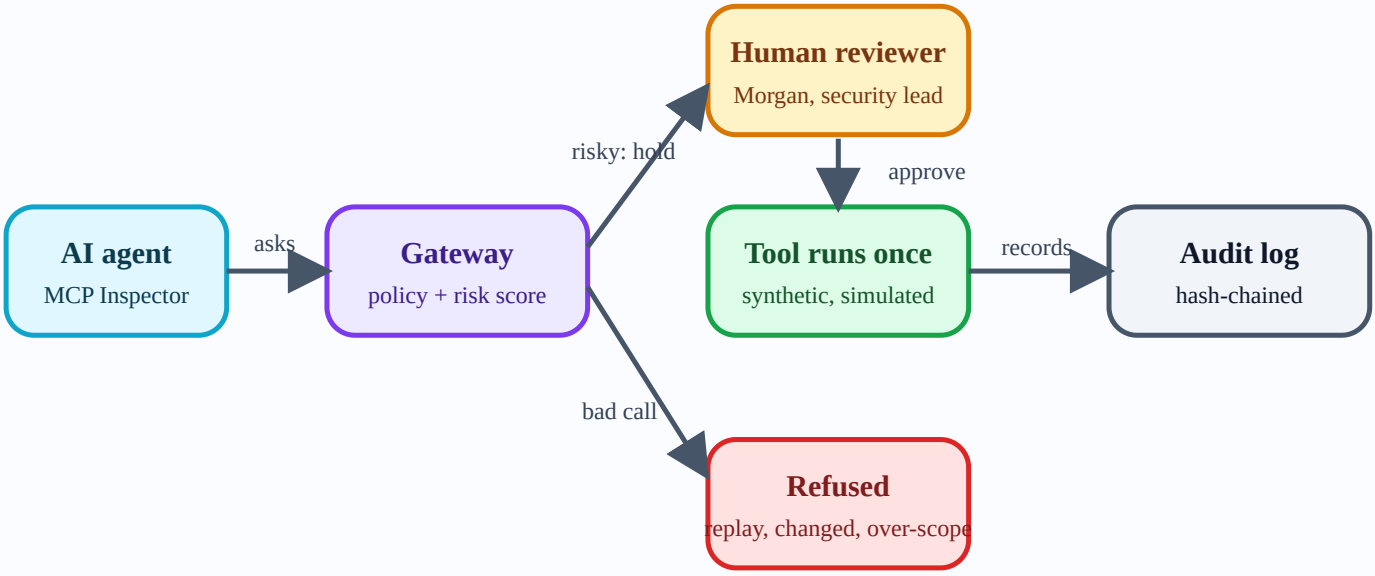
What each test proves is stated on its page. What was not proved is listed in section 5.

© 2026 Roshan Trivedi. All rights reserved.

2.1 Use case

An engineering agent is asked to ship a build to production. It connects to the gateway over MCP and asks to run `deploy.production` against `service://payment-api`. A person should decide that, not the agent. After the person says yes, the agent must only be able to do *exactly* what was approved, once.

2.2 Flow under test



Low-risk calls skip the human and run at once (T3). Risky calls wait (T4, T5). Anything that does not match what was reviewed is refused (T7, T8, T11).

2.3 Success looks like

- Safe calls stay fast; only risky ones wait for a person.
- An approval works for the exact request that was reviewed, one time.
- Every decision, including refusals, is on the record.

© 2026 Roshan Trivedi. All rights reserved.

3.1 Environment

Machine	MacBook Air (2017), macOS Monterey 12.7.6
Runtime	Node v22.23.3, npm 10.9.9 (the package asks for Node 24+, so npm printed engine warnings)
Gateway	HumanGate MCP gateway, `http://127.0.0.1:4174`, MCP endpoint `POST /mcp`, unauthenticated demo mode
Client	Official MCP Inspector v2.9.0 (`npx @modelcontextprotocol/inspector`)
Protocol	MCP 2025-11-25 negotiated (the gateway also supports 2026-07-28)
Tools	Nine, all synthetic: nothing real is read, changed or deployed
Reviewer	Morgan, security lead (lead-morgan), in the lab web page

3.2 Method

I drove the gateway from the Inspector, approved in the lab page as the reviewer, and photographed the screen at each step. Run 1 (T1 to T9) walked a risky request through the whole path. Run 2 (T10 to T13) tried changing a request after it was approved. T14 adds the project's own tests.

3.3 How to read the screenshots

1 **Green** the expected, good result

2 **Yellow** a value worth reading closely

3 **Red** a block, a refusal or a risk

The pictures are photographs of the laptop screen, so some have glare. Numbers in the marks match the key under each picture.

3.4 Test matrix

ID	Test	Expected	Result
T1	Install, build, start	Runs on a home laptop	Pass
T2	Real client connects	9 tools listed	Pass
T3	Low-risk call	Runs, no human	Pass
T4	Risky call	Held, not executed	Pass
T5	Human approval	Approved, hash matches	Pass
T6	Run with approval	Executes once	Pass
T7	Reuse approval	replay_blocked	Pass
T8	Extra scope	Denied by policy	Pass
T9	Audit log	Chain verifies	Pass
T10	Fresh request approved	Pending, then approved	Pass
T11	Changed target	binding_mismatch	Pass
T12	Original request	Executes once	Pass
T13	Refusal logged	Event in the log	Pass
T14	Automated tests	All pass	34/34

© 2026 Roshan Trivedi. All rights reserved.

T1 Install, build and start the gateway

PASS

OBJECTIVE

Show that the gateway installs, builds and starts on an ordinary home laptop.

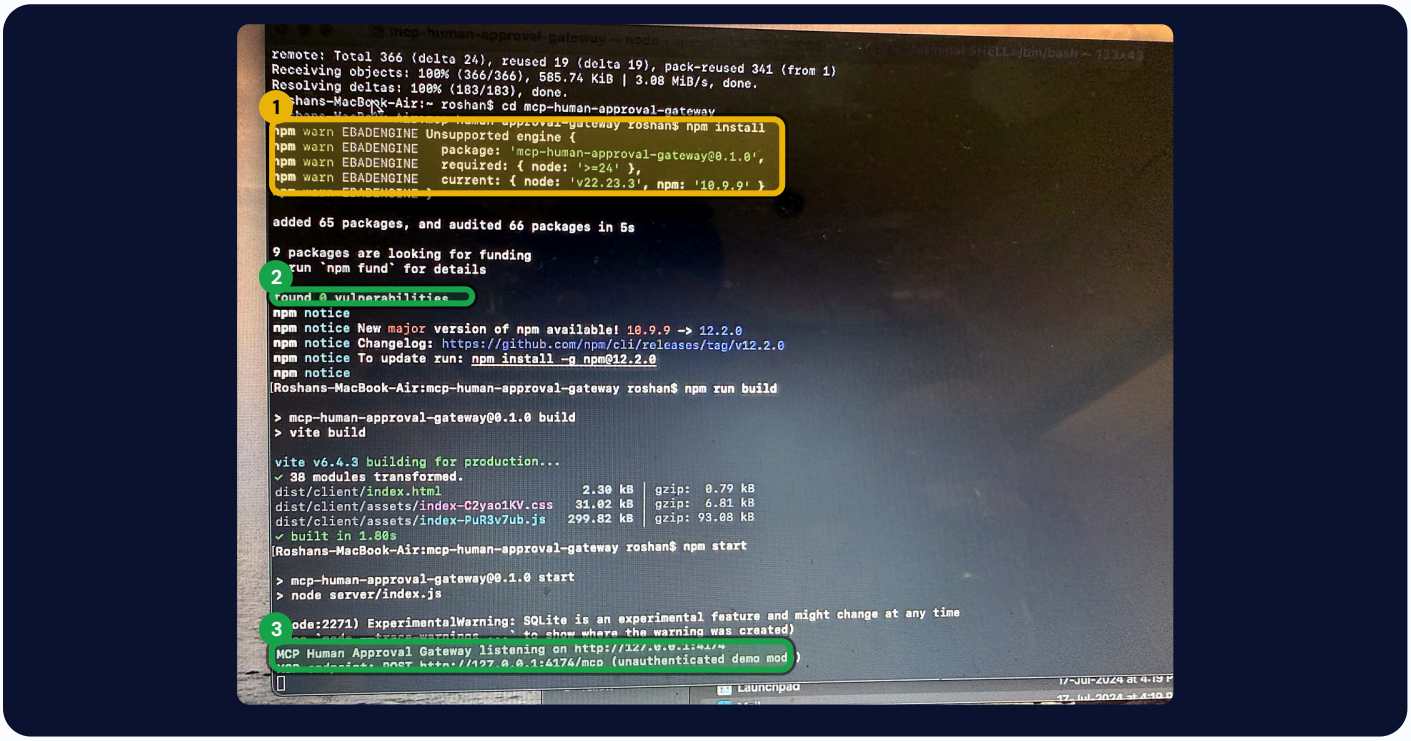
STEPS

1. Open Terminal on the MacBook Air and go into the project folder.
2. Run `npm install`, then `npm run build`, then `npm start`.
3. Read the last lines for the address the server listens on.

EXPECTED

Build finishes with no errors and the server prints its address.

SCREENSHOT



1 **Yellow** Engine warning: package wants Node 24+, laptop has v22.23.3

2 **Green** 0 vulnerabilities found

3 **Green** Gateway is listening on 127.0.0.1:4174

OUTPUT

- found 0 vulnerabilities
- vite build · built in 1.80s
- MCP Human Approval Gateway listening on http://127.0.0.1:4174
- MCP endpoint: POST http://127.0.0.1:4174/mcp (unauthenticated demo mode)

The package asks for Node 24 or newer. This laptop runs Node v22.23.3, so npm printed EBADENGINE warnings. It still built and ran.

Why it matters If a governance gate needs a data centre to try, nobody tries it. This one runs on a 2017 laptop.

© 2026 Roshan Trivedi. All rights reserved.

T2 A real MCP client connects and lists the tools

PASS

OBJECTIVE

Connect a client I did not write and confirm it can discover the gateway's tools.

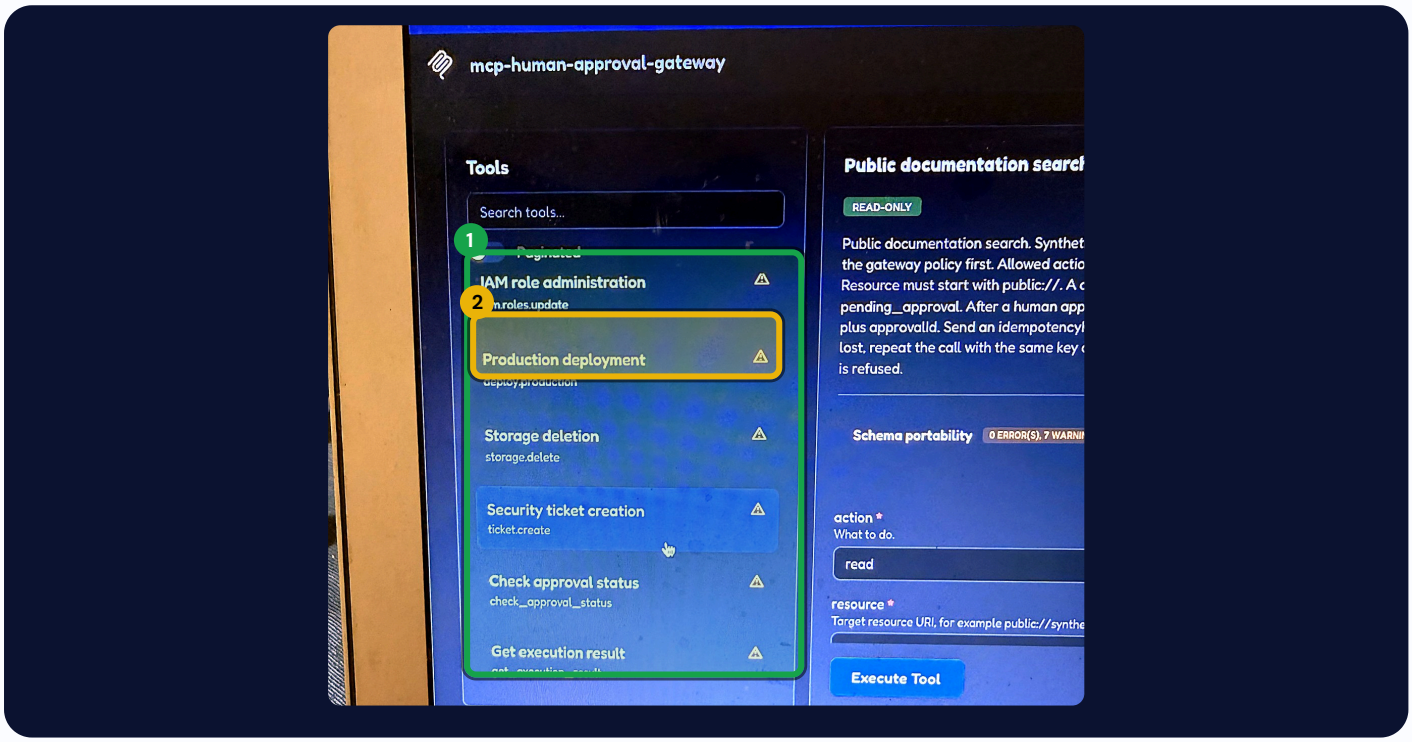
STEPS

1. Start the official MCP Inspector with `npx @modelcontextprotocol/inspector`.
2. Connect it to `http://127.0.0.1:4174/mcp`.
3. Open the Tools tab and run List Tools.

EXPECTED

Client connects, negotiates a protocol version and receives the tool definitions.

SCREENSHOT



1 **Green** Tool list received from the gateway

2 **Yellow** deploy.production, the tool used for the main tests

OUTPUT

```
• Client: MCP Inspector v2.9.0
• Protocol negotiated: MCP 2025-11-25
• tools/list returned 9 synthetic tools, including deploy.production (shown with a warning
  triangle)
```

Nine tools: docs.search, repo.read, secrets.read, iam.roles.update, deploy.production, storage.delete, ticket.create, check_approval_status, get_execution_result.

Why it matters The client is a standard tool I did not write, so the gateway has to work with what the MCP standard defines, not with my own script.

© 2026 Roshan Trivedi. All rights reserved.

T3 Low-risk request runs straight away

PASS

OBJECTIVE

Show that safe, read-only calls are not slowed down by a human.

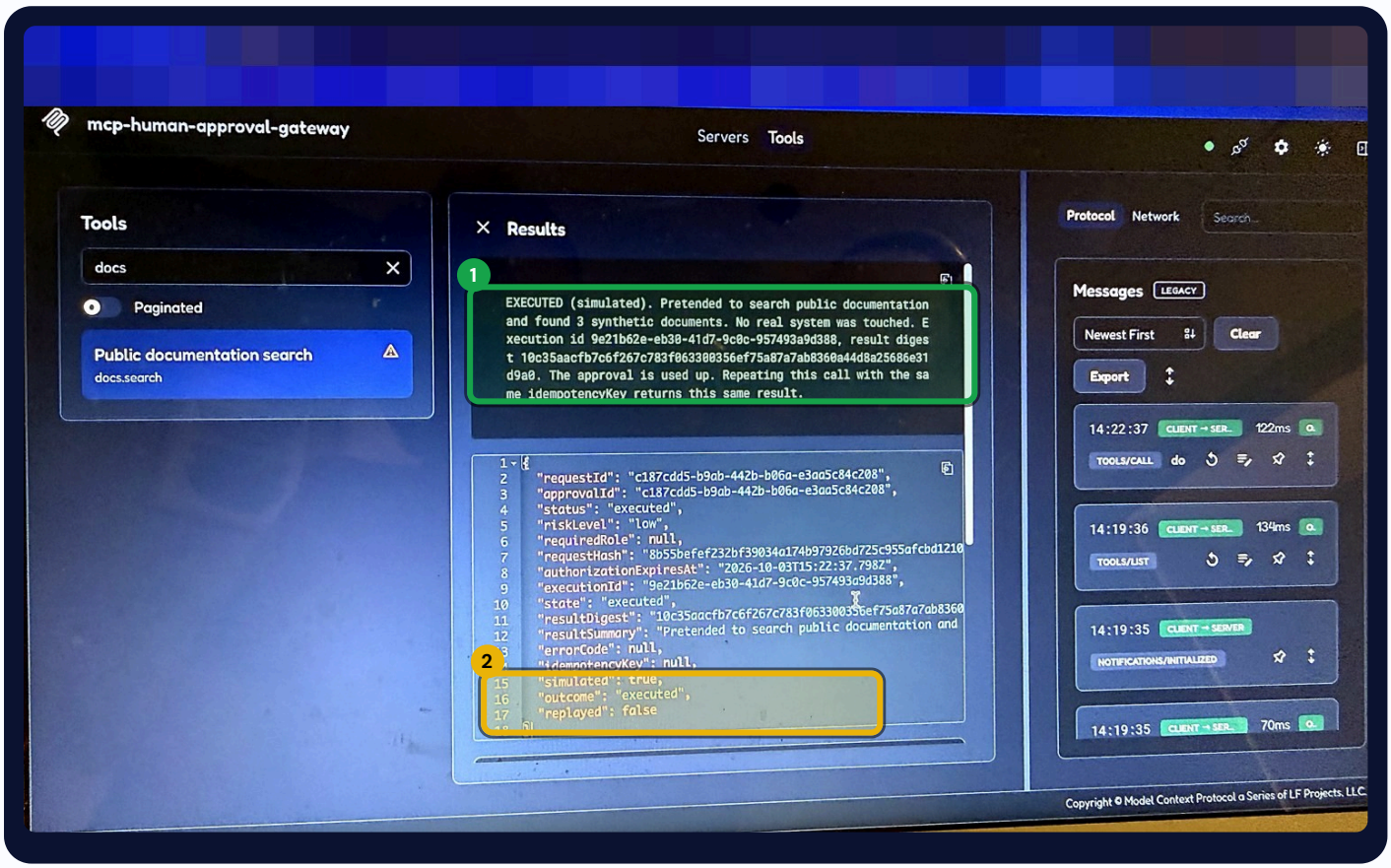
STEPS

1. In the Inspector choose docs.search (public documentation search).
2. Action read, a public:// resource, minimal scope. Click Execute Tool.

EXPECTED

Policy scores the call as low risk and lets it run with no approval.

SCREENSHOT



1 **Green** Executed at once, flagged as simulated

2 **Yellow** outcome: executed, replayed: false

OUTPUT

- `EXECUTED (simulated). Pretended to search public documentation and found 3 synthetic documents. No real system was touched.`
- `riskLevel: low · outcome: executed · replayed: false`

Why it matters Controls that slow every call get switched off. Safe calls have to stay fast.

© 2026 Roshan Trivedi. All rights reserved.

T4 Risky request is held for a human

PASS

OBJECTIVE

Show that a production deployment does not run on the agent's say-so.

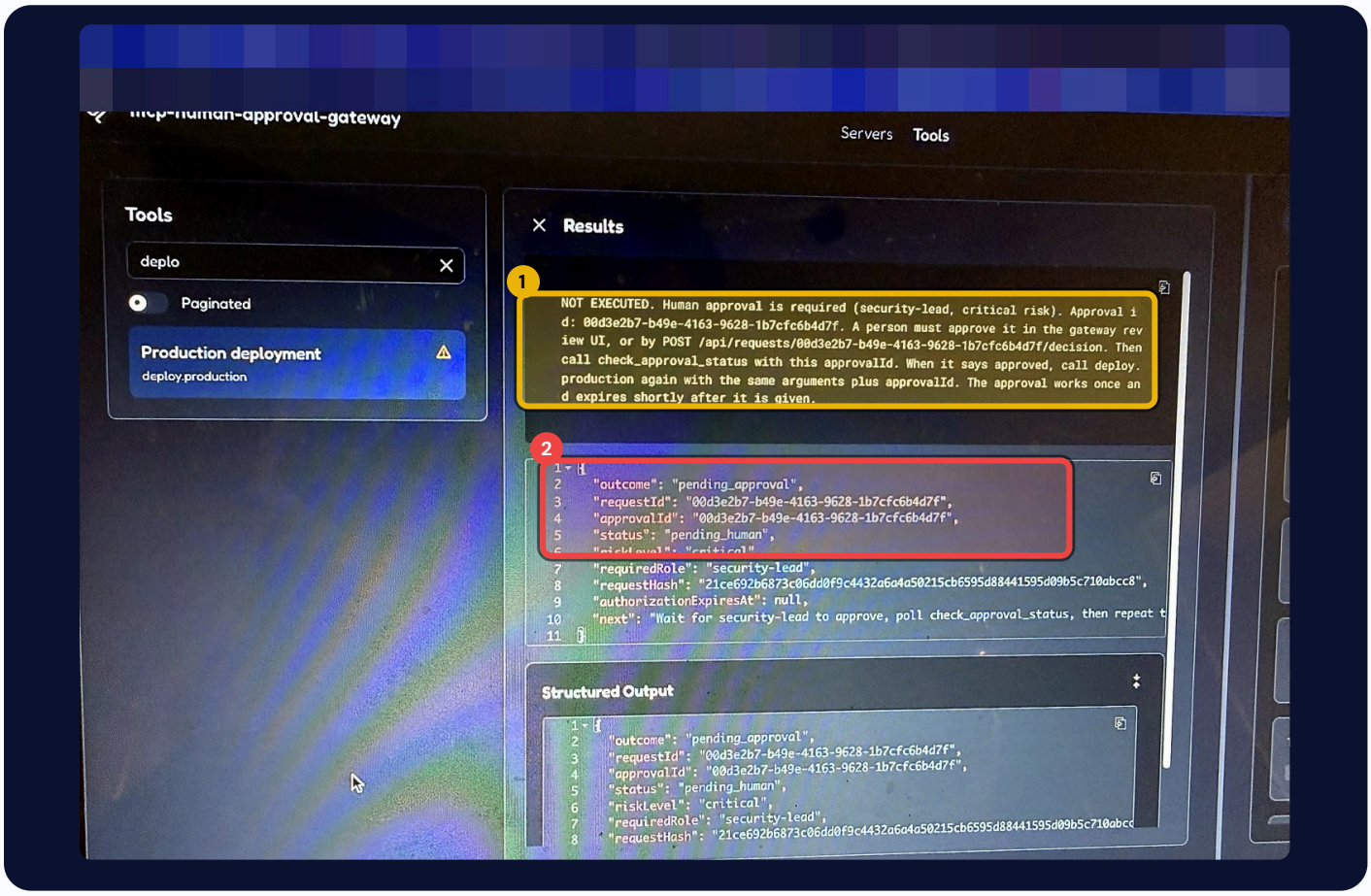
STEPS

1. Choose deploy.production. Action deploy, resource `service://payment-api`, environment production, scope `deploy:production`.
2. Click Execute Tool.

EXPECTED

Policy scores it critical and returns pending_approval. Nothing is executed.

SCREENSHOT



1 **Yellow** Held: NOT EXECUTED, human approval required

2 **Red** pending_approval, critical, security-lead needed

OUTPUT

- NOT EXECUTED. Human approval is required (security-lead, critical risk).
- outcome: pending_approval · status: pending_human · riskLevel: critical · requiredRole: security-lead
- requestHash 21ce692b…0abcc8 · authorizationExpiresAt: null

Why it matters An agent one prompt away from a production change needs a gate it cannot talk its way past.

© 2026 Roshan Trivedi. All rights reserved.

T5 A security lead approves in the review screen

PASS

OBJECTIVE

Show that the decision is made by a person with the right role, in a separate screen.

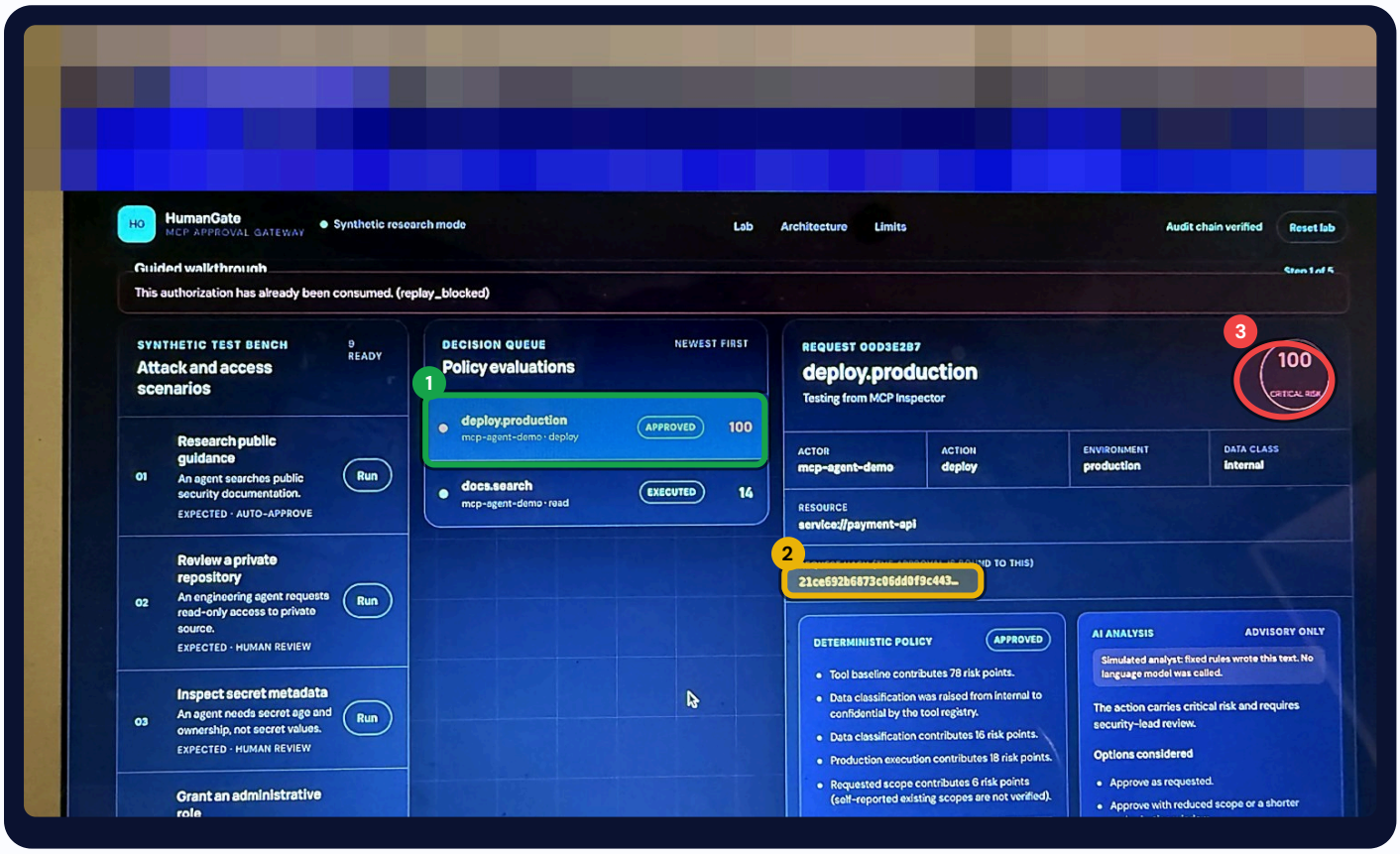
STEPS

1. Open the lab page at `localhost:4174`.
2. Pick the pending deploy.production request in the decision queue.
3. Reviewer Morgan (security lead), type a rationale, click Approve.

EXPECTED

Request shows Approved. The request hash matches the one the agent received.

SCREENSHOT



1 **Green** Approved by the human reviewer

2 **Yellow** Same request hash as T4

3 **Red** Risk score 100, critical

OUTPUT

```
• deploy.production · APPROVED · score 100
• Request hash 21ce692b6873c06d… (same as in T4)
• docs.search from T3 shown as EXECUTED, score 14
```

Why it matters The decision sits with a person who has the right role, in a screen the agent cannot reach.

© 2026 Roshan Trivedi. All rights reserved.

T6 The approved request runs exactly once

PASS

OBJECTIVE

Show that the approval lets the exact reviewed request through, one time.

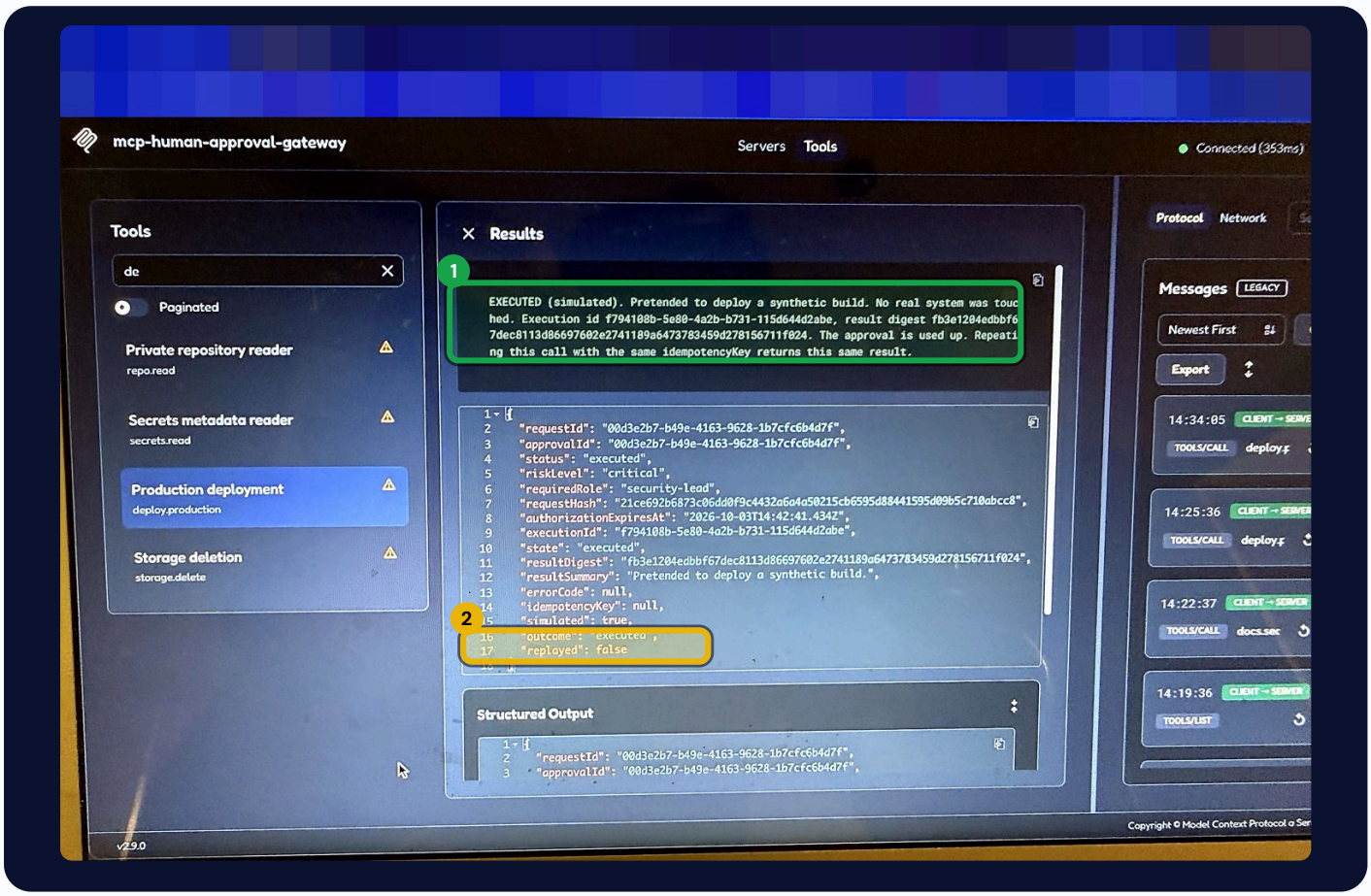
STEPS

1. Call deploy.production again with identical arguments plus the approvalId.
2. Click Execute Tool.

EXPECTED

The action executes (simulated) and the approval is marked used.

SCREENSHOT



1 **Green** Executed once; approval used up

2 **Yellow** outcome: executed, replayed: false

OUTPUT

```
• EXECUTED (simulated). Pretended to deploy a synthetic build. No real system was touched.
• The approval is used up.
• outcome: executed · replayed: false
```

Why it matters The approval is a single-use pass for one specific action, not a standing permission.

© 2026 Roshan Trivedi. All rights reserved.

T7 Reusing the approval is refused

PASS

OBJECTIVE

Show that one approval cannot be replayed to run the action again.

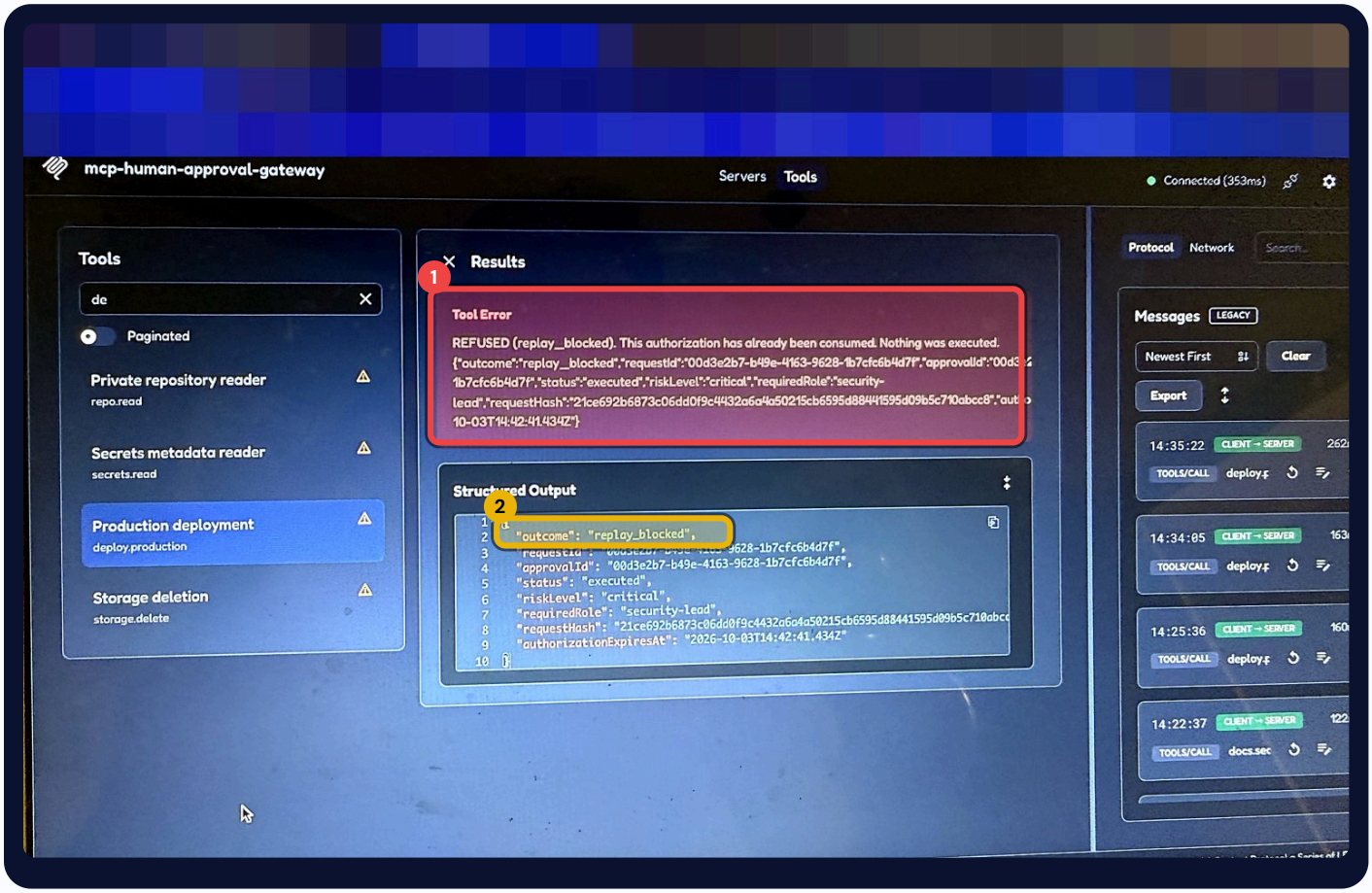
STEPS

1. Send the same call again with the same approvalId.

EXPECTED

Refused. Nothing runs a second time.

SCREENSHOT



1 **Red** Refused: approval already consumed

2 **Yellow** outcome: replay_blocked

OUTPUT

```
• REFUSED (replay_blocked). This authorization has already been consumed. Nothing was executed.
• outcome: replay_blocked
```

Why it matters If an approval could be replayed, one yes would turn into unlimited access.

© 2026 Roshan Trivedi. All rights reserved.

T8 Asking for extra permission is denied by policy

PASS

OBJECTIVE

Show that a call cannot quietly ask for more access than its tool is registered for.

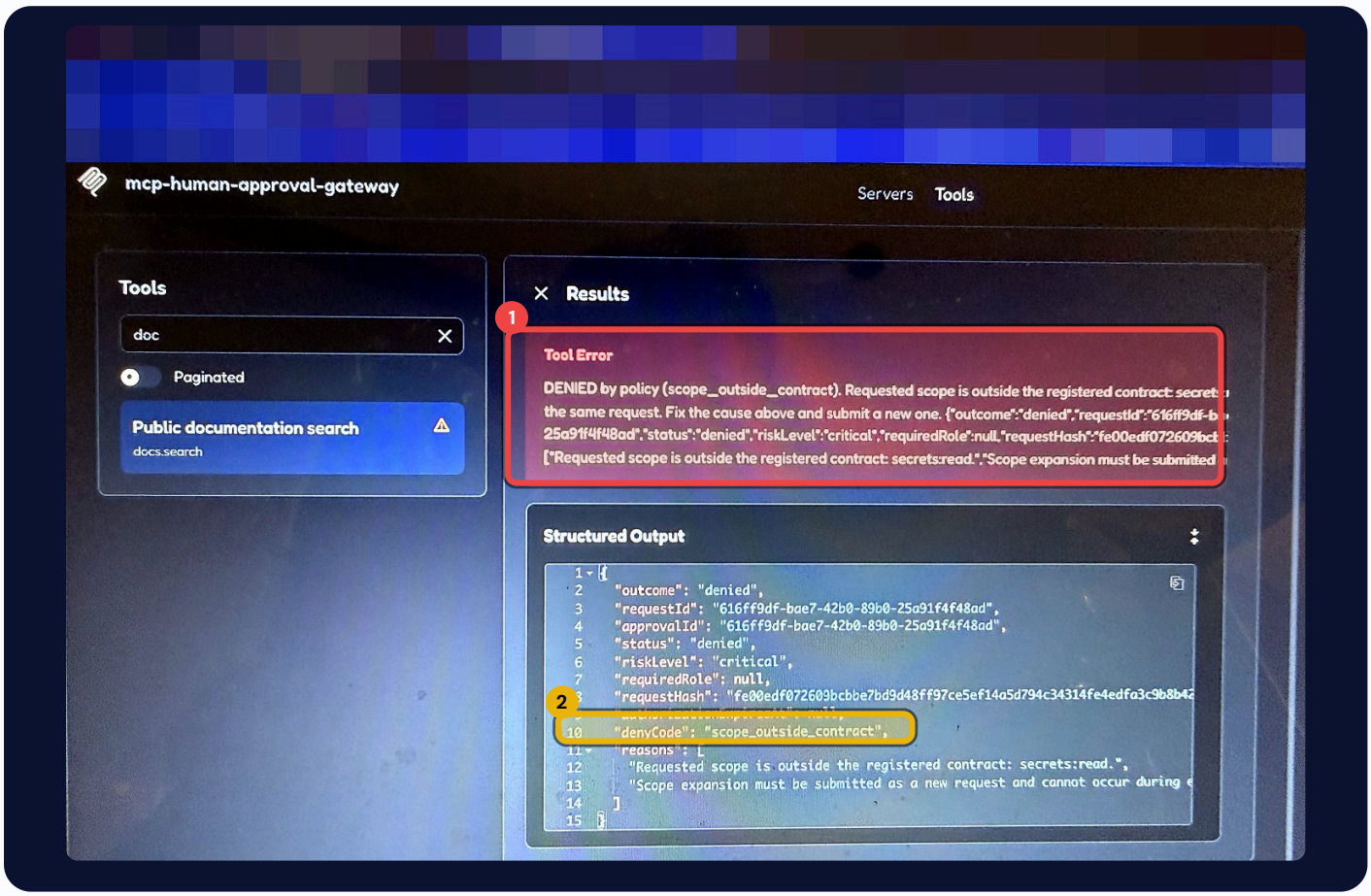
STEPS

1. Choose docs.search and add `secrets:read` to the requested scopes.
2. Click Execute Tool.

EXPECTED

Policy denies it. No human is asked, nothing runs.

SCREENSHOT



1 **Red** Denied: scope outside the registered contract

2 **Yellow** denyCode: scope_outside_contract

OUTPUT

- `DENIED by policy (scope_outside_contract). Requested scope is outside the registered contract: secrets:read.`
- `denyCode: scope_outside_contract · outcome: denied`

Why it matters Access cannot grow quietly inside a request. More scope means a new request and a new review.

© 2026 Roshan Trivedi. All rights reserved.

T9 Every step is in a tamper-evident log

PASS

OBJECTIVE

Show that the decisions are recorded in a hash-chained log that can be verified.

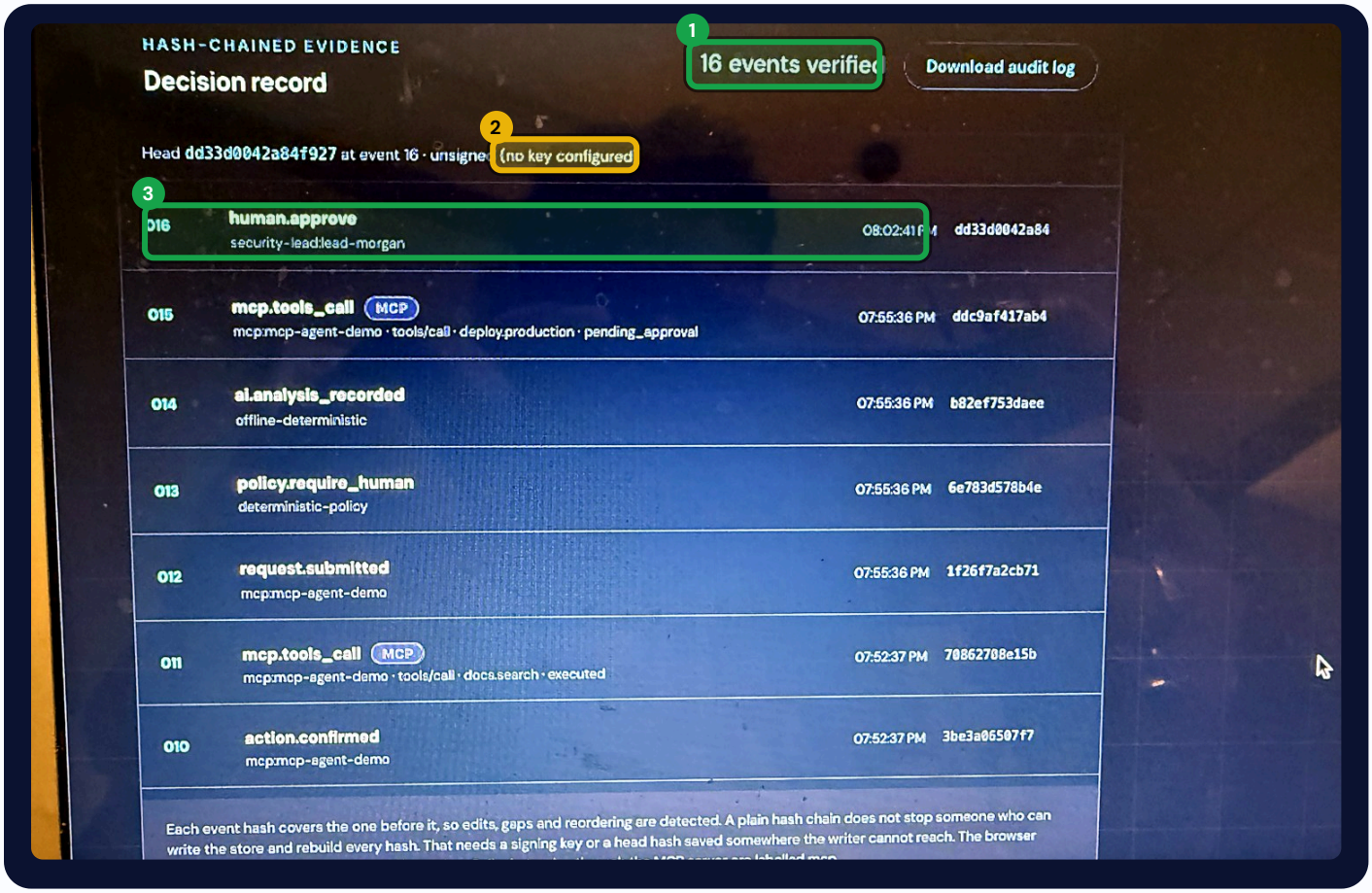
STEPS

1. Scroll to the Decision record in the lab page.

EXPECTED

Events are listed, calls from the client are labelled MCP, and the chain verifies.

SCREENSHOT



1 **Green** Chain verified

2 **Yellow** Unsigned: no key configured

3 **Green** Human approval on record

OUTPUT

```
• 16 events verified at this point · head dd33d0042a84f927 · unsigned (no key configured)
• human.approve by security-lead:lead-morgan
• mcp.tools_call (MCP) deploy.production pending_approval · docs.search executed · action.confirmed
• Exported log at the end of the session: 26 events, valid, unsigned.
```

The log is a plain hash chain with no signing key. Someone who can write the store could rebuild it, so treat it as an illustration.

Why it matters When something goes wrong there is a record of who allowed what, and edits to that record can be detected.

© 2026 Roshan Trivedi. All rights reserved.

T10 New test: fresh request held, then approved

PASS

OBJECTIVE

Set up the changed-request test with a fresh request and a human approval.

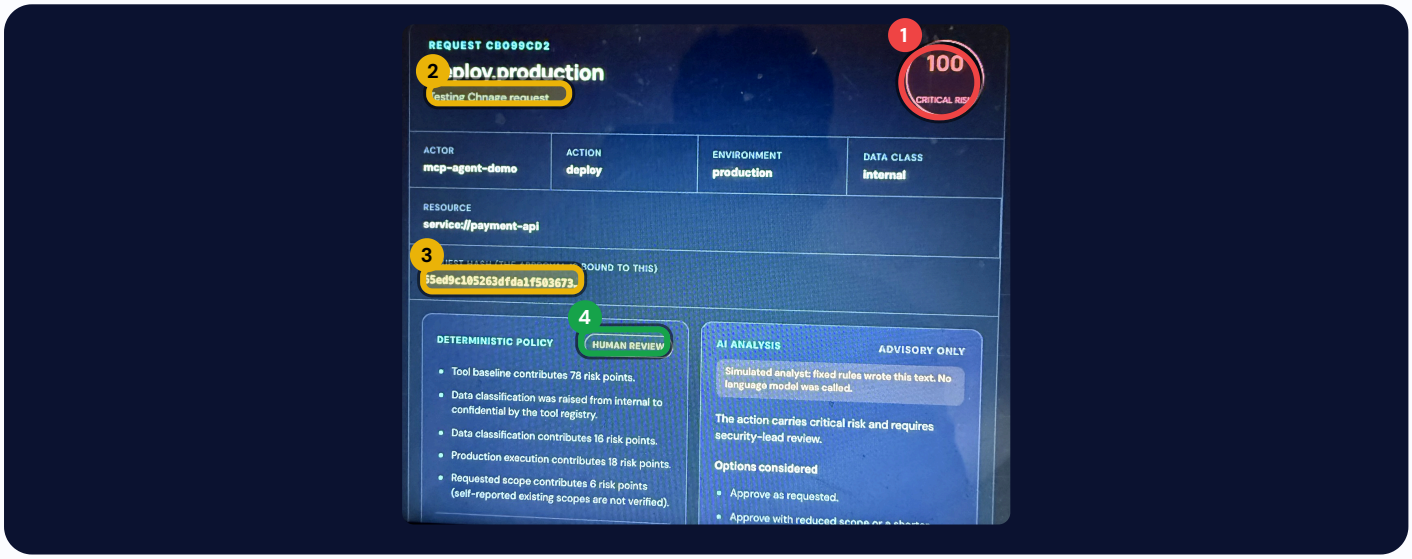
STEPS

1. Call deploy.production for `service://payment-api` with the justification `Testing Chnage request` (the typo is kept on purpose: the justification is part of the hash).
2. In the lab, Morgan (security lead) approves with the rationale "Human Review".

EXPECTED

Pending first, then Approved. The approval is bound to a hash of every field.

SCREENSHOTS

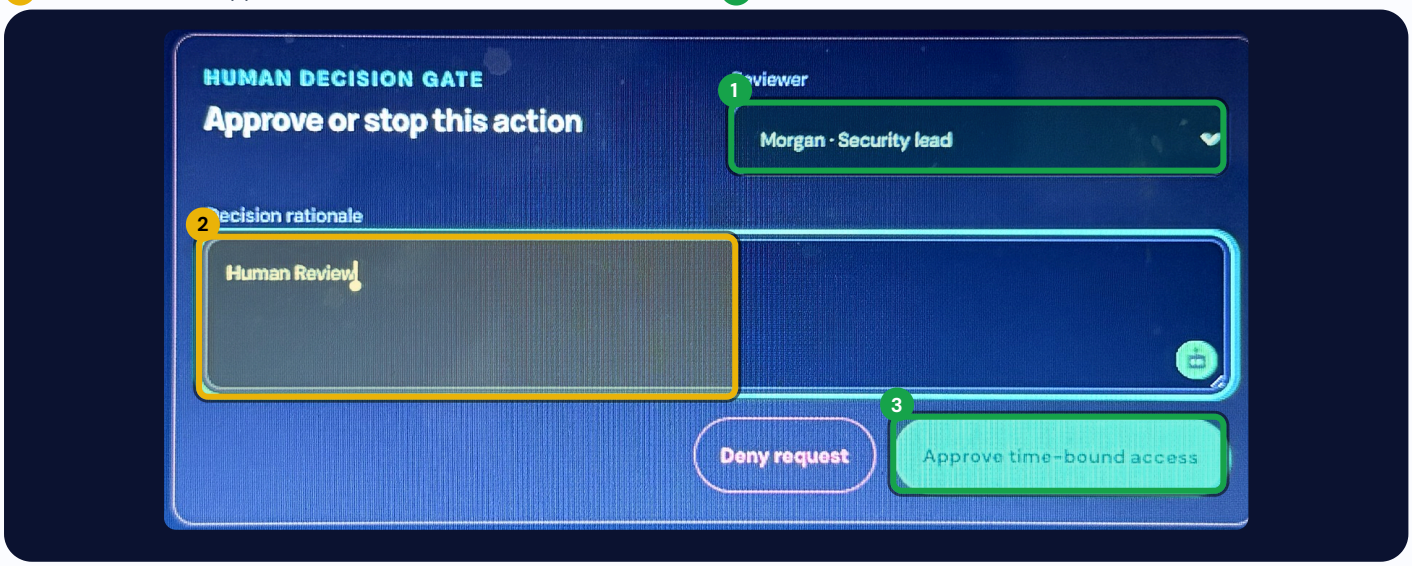


1 **Red** Risk 100, critical

2 **Yellow** Justification, part of the hash

3 **Yellow** Hash the approval is bound to

4 **Green** Held for human review



1 **Green** Reviewer: Morgan, security lead

2 **Yellow** Decision rationale

3 **Green** Approve time-bound access

OUTPUT

```
• REQUEST CB099CD2 · deploy.production · score 100 · CRITICAL RISK · HUMAN REVIEW
• requestHash 65ed9c10…58c5b3c9 · approvalId cb099cd2-5212-47dc-a97c-73757ad6ed27
```

© 2026 Roshan Trivedi. All rights reserved. roshantrivedi.co.in

T11 Same approval, changed target: refused

PASS

OBJECTIVE

Show that an approval cannot be reused for a different request. This is the main test of this report.

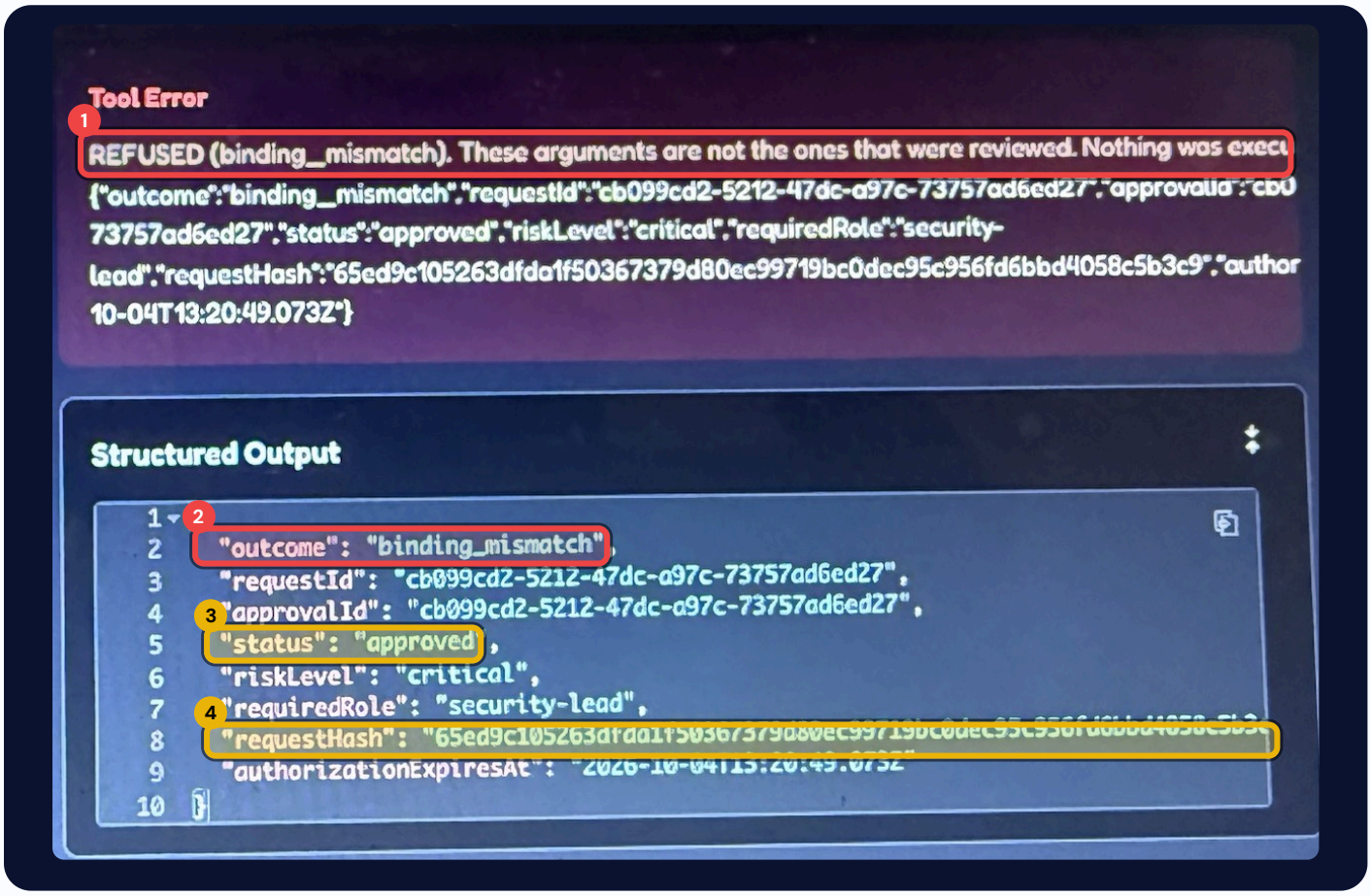
STEPS

1. Call deploy.production again with the same approvalId, but change the resource to `service://payment-api-v2` .
2. Leave every other field as it was. Click Execute Tool.

EXPECTED

Refused with binding_mismatch. The approval is not consumed.

SCREENSHOT



1 **Red** Refused: not the arguments that were reviewed

2 **Red** outcome: binding_mismatch

3 **Yellow** Approval still approved, not used up

4 **Yellow** Hash of the reviewed request

OUTPUT

- `REFUSED (binding_mismatch). These arguments are not the ones that were reviewed. Nothing was executed.`
- `status: approved (unchanged) · requestHash 65ed9c10…58c5b3c9 (the reviewed one)`

Why it matters This is the attack that matters: get a harmless request approved, then swap the target. The gateway notices because the hash no longer matches.

© 2026 Roshan Trivedi. All rights reserved.

T12 The exact reviewed request still runs once

PASS

OBJECTIVE

Show that the approval was not damaged by the failed attempt and works for the request it was given for.

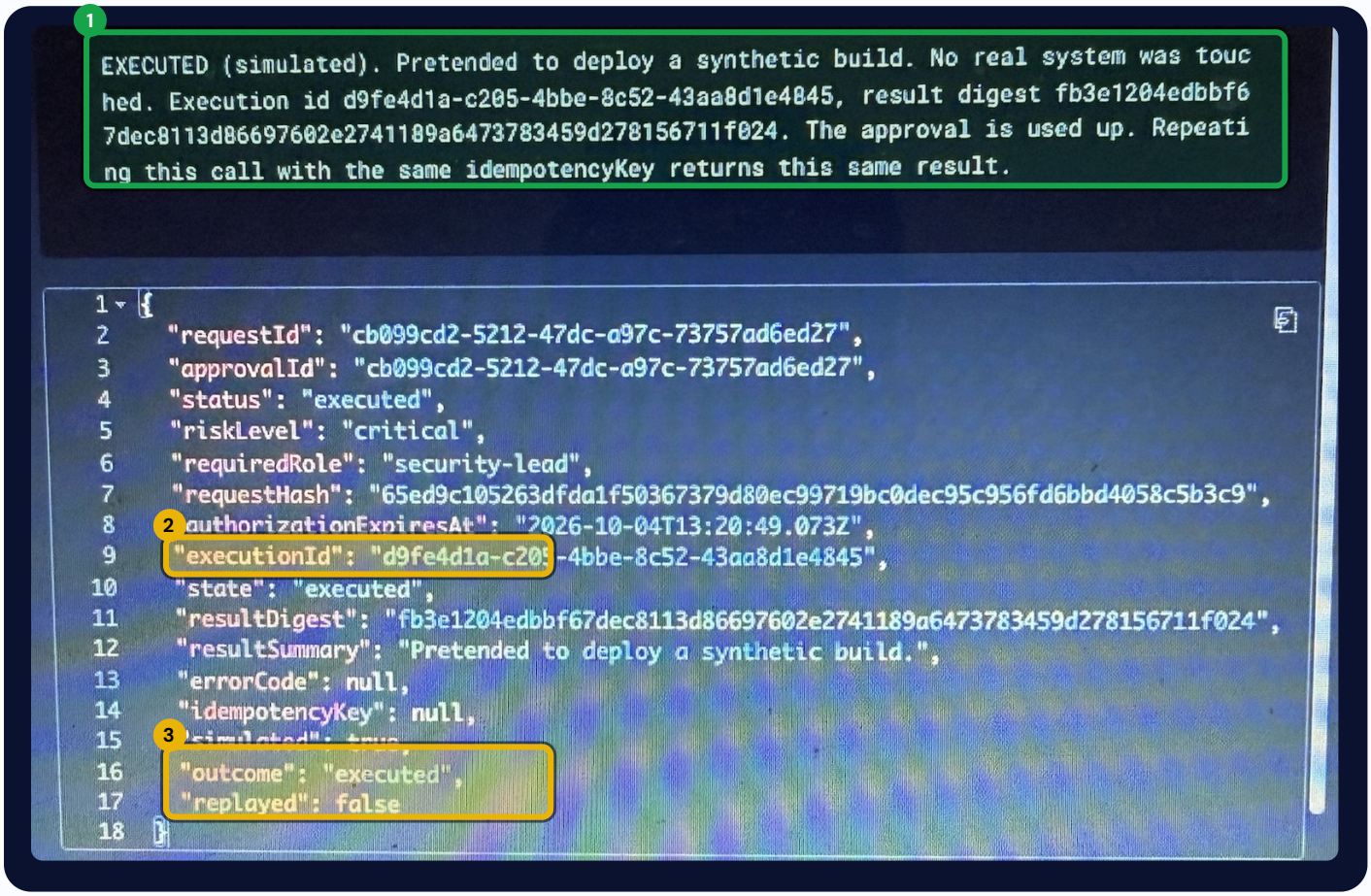
STEPS

1. Call deploy.production again with the original, unchanged arguments and the same approvalId.
2. Click Execute Tool.

EXPECTED

Executes once (simulated) and the approval is used up.

SCREENSHOT



1 **Green** Executed once; approval used up

2 **Yellow** state: executed

3 **Yellow** outcome: executed, replayed: false

OUTPUT

- EXECUTED (simulated). Pretended to deploy a synthetic build. No real system was touched.
- executionId d9fe4d1a-c205-4bbe-8c52-43aa8d1e4845 · The approval is used up.
- state: executed · outcome: executed · replayed: false

Why it matters Shows the refusal did not break the legitimate path. The right request still goes through, once.

© 2026 Roshan Trivedi. All rights reserved.

T13 The refusal is on the record

PASS

OBJECTIVE

Show that the refused attempt was logged as its own event.

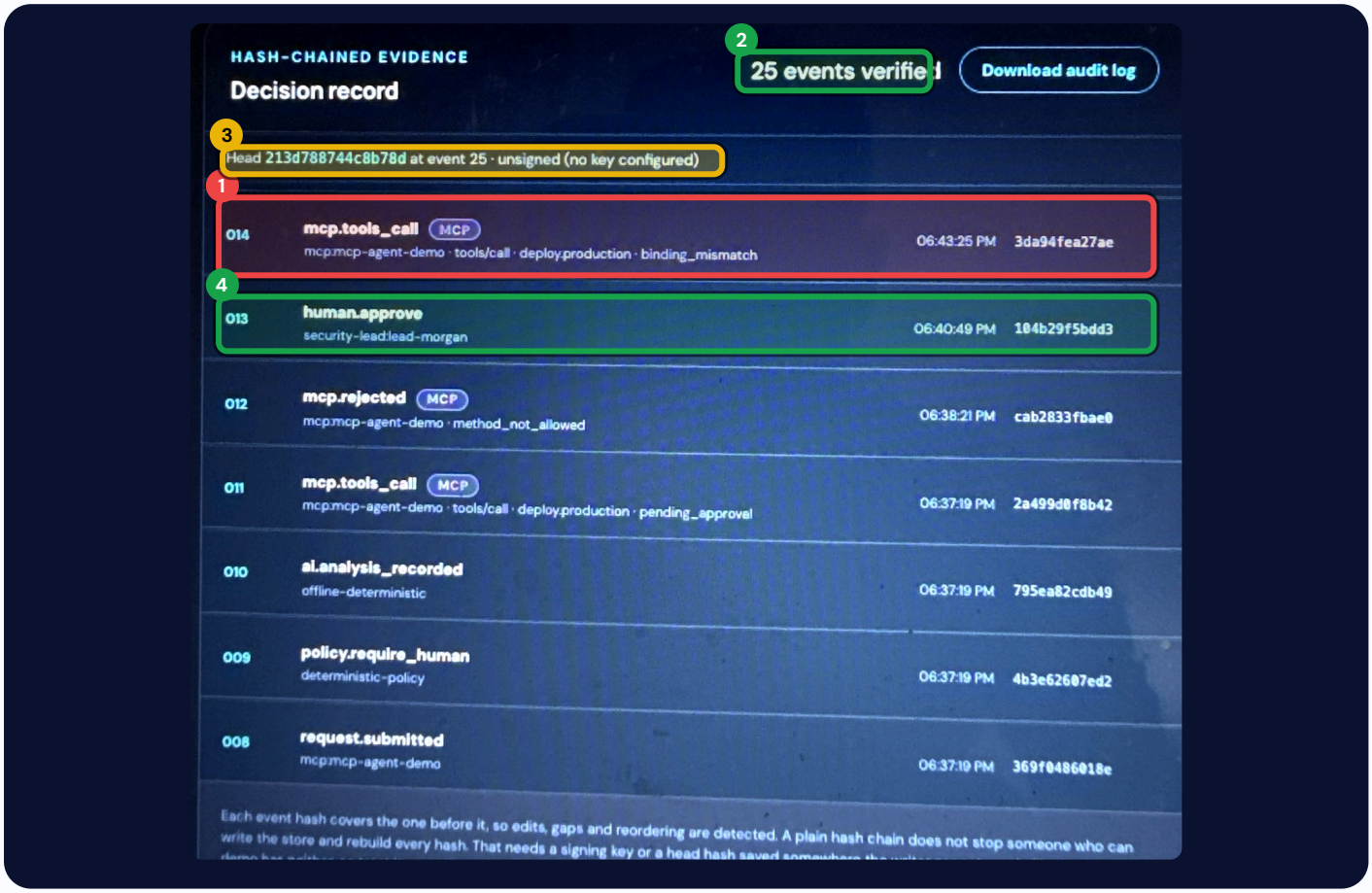
STEPS

1. Open the Decision record in the lab page after T11.

EXPECTED

A binding_mismatch event appears and the chain still verifies.

SCREENSHOT



1 **Red** The refused attempt, logged

2 **Green** Chain verified

3 **Yellow** Unsigned log

4 **Green** The human approval before it

OUTPUT

```
• 25 events verified · head 213d788744c8b78d · unsigned (no key configured)
• 014 mcp.tools_call · tools/call · deploy.production · binding_mismatch (06:43:25 PM)
• 013 human.approve · security-lead:lead-morgan (06:40:49 PM)
```

Why it matters A refused attempt is evidence. Having it logged helps anyone reviewing an incident later.

© 2026 Roshan Trivedi. All rights reserved.

T14 The project's own automated tests

PASS

OBJECTIVE

Run the automated MCP tests on the same laptop as a second layer of evidence.

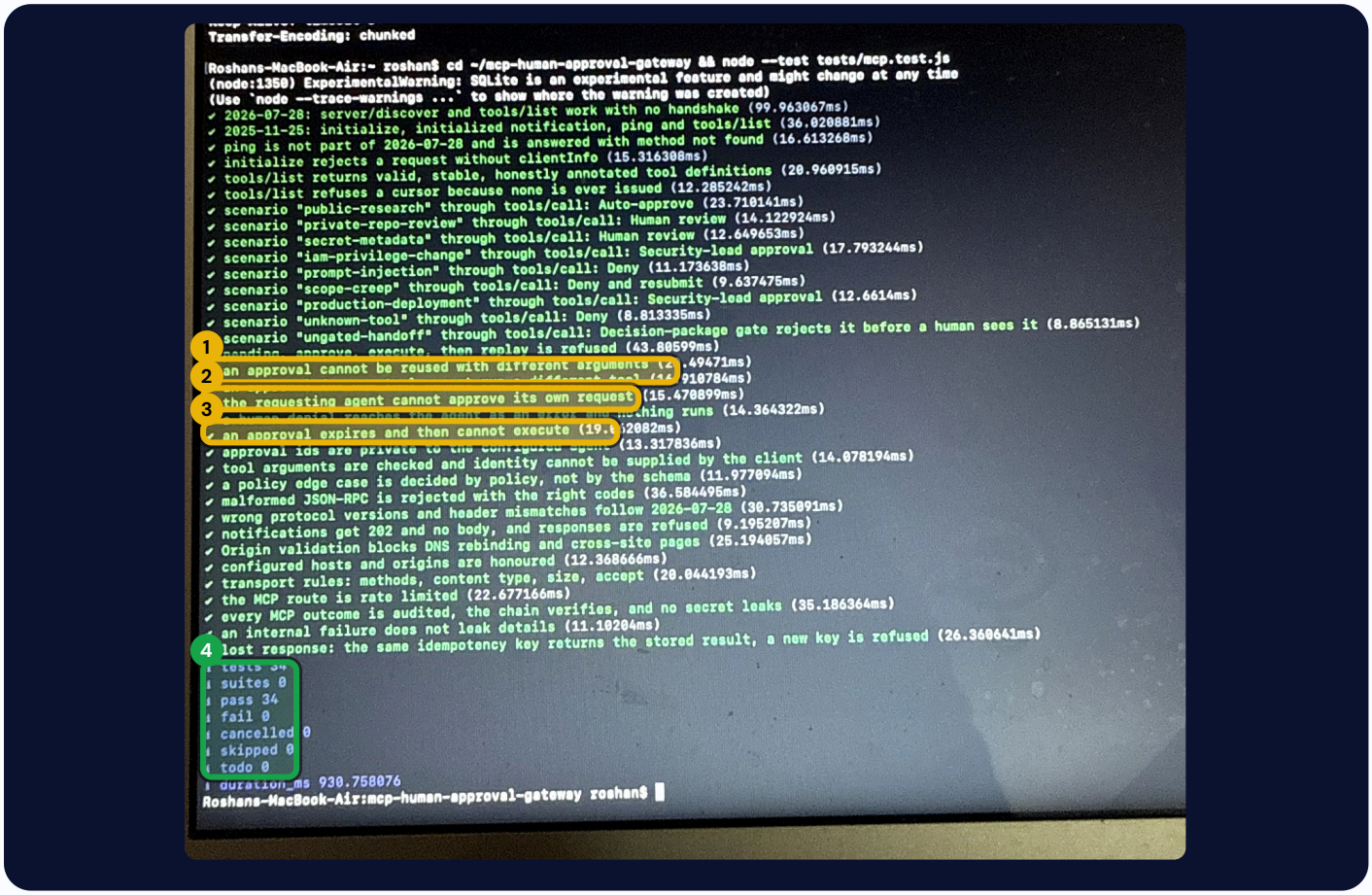
STEPS

1. Run `node --test tests/mcp.test.js` in the project folder.

EXPECTED

All tests pass.

SCREENSHOT



1 **Yellow** Approval cannot be reused with different arguments

2 **Yellow** Requesting agent cannot approve its own request

3 **Yellow** Approval expires, then cannot execute

4 **Green** 34 of 34 pass, 0 fail

OUTPUT

- tests 34 · pass 34 · fail 0 · cancelled 0 · skipped 0 · duration 930 ms
- Includes: an approval cannot be reused with different arguments; the requesting agent cannot approve its own request; an approval expires and then cannot execute.

These are the project's own tests. Self-approval and expiry were not tried by hand with a real client (see Limits).

Why it matters Hand-run tests show it works once. Automated tests show it keeps working as the code changes.

© 2026 Roshan Trivedi. All rights reserved.

4 Results at a glance

Question	What happened
Do safe calls stay fast?	**Yes. docs.search ran at once (risk 14).**
Is a risky call held?	**Yes. deploy.production scored 100 and returned pending_approval.**
Does a human decide?	**Yes. Morgan, security lead, approved in a separate screen.**
Does the approval work once?	**Yes. Executed once; reuse gave replay_blocked.**
Can the request change after approval?	**No. Changing the target gave binding_mismatch; the original request still ran once.**
Can a call ask for extra access?	**No. secrets:read was denied by policy.**
Is it all on record?	**Yes. Chain verified; the refusal is its own event.**

5 Limits and what was not tested

- **Synthetic only.** All nine tools pretend to act. This shows the gate, not real deployments.
- **One client.** Only the MCP Inspector was run by hand. Claude Code and others are not yet tested.
- **Not tried by hand:** an agent approving its own request, an approval expiring on a clock, and the `check_approval_status` tool. The project's automated tests cover the first two (T14).
- **Demo mode.** No real user login. Reviewer names are plain strings in the lab.
- **Unsigned log.** The hash chain catches edits, gaps and reordering, but not someone who can rewrite the whole store. A signing key or an outside copy of the head hash is needed.
- **Screenshots are photos** of a laptop screen, so some have glare.

> Not a security certification or a production-readiness claim. It is a lab result.

© 2026 Roshan Trivedi. All rights reserved.

6 What is next

Second client

Run Claude Code against the gateway and repeat the key tests.

Hand-run the gaps

Self-approval, expiry and check_approval_status with a real client.

Stronger evidence

Sign the audit log and add real reviewer login.

7 About the author and contact

Roshan Trivedi

Leader in IAM and PAM, working on AI use cases across identity: AI bots, agentic AI, generative AI, MCP security, authentication and usage. Based in Bengaluru, India.

WEBSITE
roshantrivedi.co.in

LINKEDIN
linkedin.com/in/roshan-trivedi-ciam-49413a54

SOURCE CODE
github.com/trivediroshan1-ui/mcp-human-approval-gateway

EMAIL
trivedi.roshan1@gmail.com

Website

LinkedIn

8 Copyright and notice

© 2026 Roshan Trivedi. All rights reserved. This report, including its text, layout, diagrams and annotated screenshots, is the work of Roshan Trivedi. Please credit the author if you share or quote it.

The HumanGate gateway source code is a separate work published under the MIT licence in the repository above. This report does not change that licence.

The results describe a lab with synthetic tools on one laptop on the dates shown. They are shared for information only and are not a warranty, a security audit or advice. Product names belong to their owners. “MCP” refers to the Model Context Protocol.